

Computing Methods for Physics 1 – 19 June 2024

Your exam material (code files, plots, datafiles, etc.) must be submitted via google classroom by 13:30 as a single zip file.

C++ evaluation will be based on: correct syntax, proper return types, proper arguments of functions, data members and class interfaces, setters/getters, unnecessary void functions, correct mathematical expressions, comments throughout the code, separation of class implementations and interfaces.

Python evaluation will be based on: correct syntax, avoiding C-style loops, using Python features in general, comments throughout the notebook/scripts, labels, legends and plot styling, clarity in general.

Part 1 – C++

Consider an ideal gas with specific heat coefficients $c_V = c_V(T)$ and $c_P = c_P(T)$, where T is the temperature. These coefficients are linked by Mayer's relation $c_P - c_V = R$, where $R = 8.314 \text{ J} \cdot \text{mol}^{-1} \cdot \text{K}^{-1}$ is the molar gas constant. Further, the quantity $\gamma = c_P/c_V$ is introduced for convenience.

Write the class `IdealGas` to represent n moles of an ideal gas and two classes that inherit from it:

1. `MonatomicIG`, a concrete class with $c_V = 3R/2$ and $c_P = 5R/2$, regardless of T ;
2. `BiatomicIG`, a concrete class with $c_V = 5R/2$ and $c_P = 7R/2$, regardless of T ;

Denoting the heat exchanged by the gas with Q and the work performed by the gas with W , the parent class `IdealGas` must provide interfaces for methods to handle the following processes between the end/start states (E/S):

- isochoric (constant volume) – $p_E/p_S = T_E/T_S$, $W = 0$, $Q = nc_V(T_E - T_S)$;
- isobaric (constant pressure) – $V_E/V_S = T_E/T_S$, $W = nR(T_E - T_S)$, $Q = nc_P(T_E - T_S)$;
- adiabatic (no heat exchange): $Q = 0$, $W = nc_V(T_E - T_S)$, $T_E V_E^{\gamma-1} = T_S V_S^{\gamma-1}$; this last equality holds only if the process is reversible, which the user must be able to specify.

In all processes, assume that the variable unknown to the user is T_E , and that, in addition to T_E , they may be interested in calculating Q and W .

Part 2 – Python

Use a Python notebook or Python scripts to complete the following tasks.

1. Load the dictionary stored in the file `data.pickle`.
2. Plot the data content of the dictionary.
3. Set up a Monte Carlo with `emcee` to fit the data.
4. Present the results of your fit with plots, prints, and comments.

Suggestions:

- use a sum of Gaussians as a model for the data;
- do not implement your own Gaussian function – use `scipy.stats.norm` instead.